



NAM CAN THO UNIVERSITY

Kiến trúc máy tính (Computer Architecture)

Chương 4 BỘ XỬ LÝ TRUNG TÂM (Central Processing Unit – CPU)



Mục đích, yêu cầu

- **Mục đích:** Giới thiệu tổ chức, nguyên lý hoạt động của các bộ phận bên trong bộ xử lý nhằm thực hiện nhiệm vụ mà kiến trúc phần mềm đã đề ra. Giới thiệu các kỹ thuật nâng cao hiệu quả hoạt động của các bộ xử lý hiện đại.
- **Yêu cầu:** Sinh viên nắm vững cấu trúc của bộ xử lý trung tâm, diễn tiến thi hành một lệnh mã máy và các kỹ thuật nâng cao hiệu quả hoạt động của các bộ xử lý như: kỹ thuật ống dẫn, siêu ống dẫn, siêu vô hướng, máy tính có lệnh thật dài, máy tính véc-tơ, xử lý song song.

4.1. Cấu trúc cơ bản của CPU

- Đường đi của dữ liệu
- Bộ điều khiển

4.2. Hoạt động của CPU

- Chu trình lệnh
- Diễn tiến thi hành lệnh mã máy
- Kỹ thuật ống dẫn

4.3. Tập lệnh của CPU

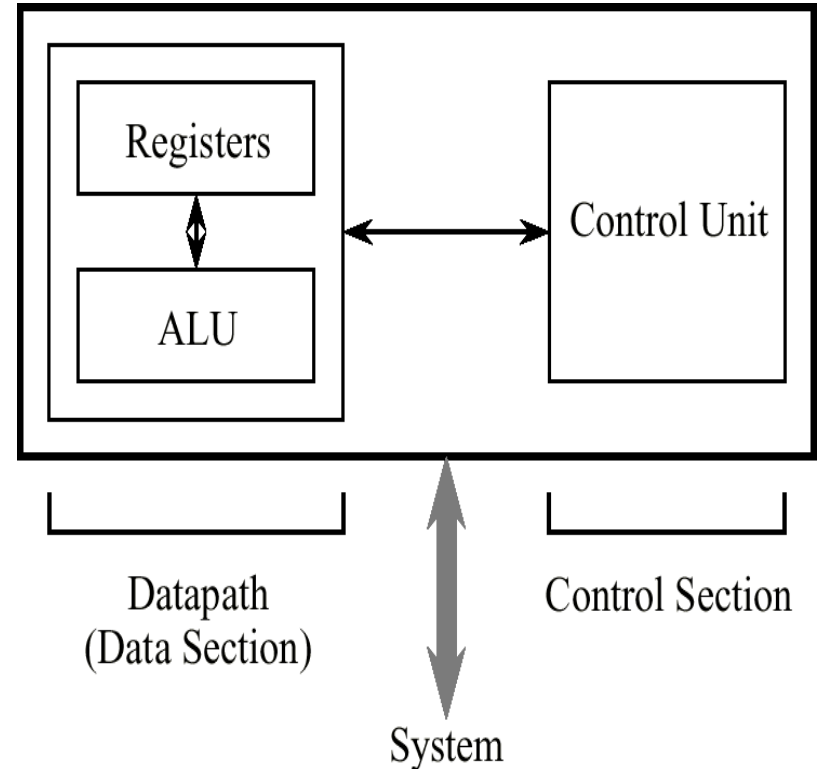
4.4. Kiến trúc của bộ xử lý tiên tiến

- Kỹ thuật siêu ống dẫn
- Kỹ thuật siêu vô hướng
- Máy tính có lệnh thật dài
- Máy tính vector
- Máy tính song song

4.5. Kiến trúc Intel – IA-64

4.1. Cấu trúc cơ bản của CPU (1)

- **Bộ xử lý trung tâm gồm:**
 - Phần thi hành lệnh (EU: Execution Unit)
Gồm: ALU & Registers
 - Bộ điều khiển (CU: Control Unit).
- Phần thi hành lệnh còn được gọi là đường đi dữ liệu (Datapath).

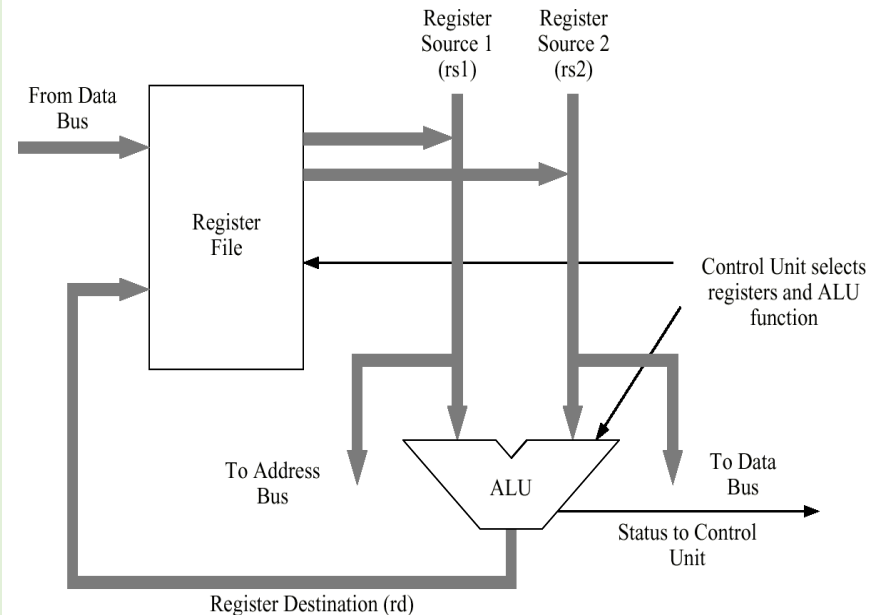


4.1. Cấu trúc cơ bản của CPU (2)

1. Đường đi của dữ liệu

• Nhiệm vụ:

- Đọc toán hạng từ các thanh ghi.
- Thực hiện các phép tính trong ALU.
- Lưu kết quả trong các thanh ghi

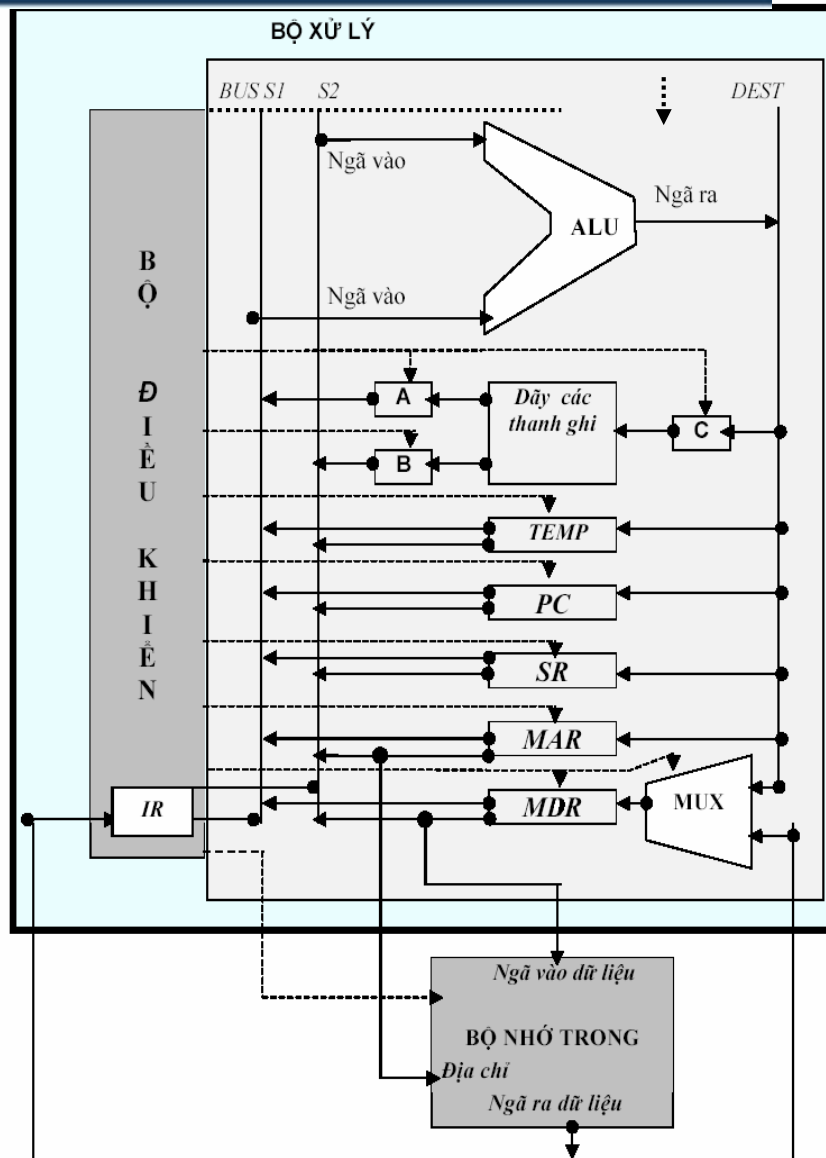


4.1. Cấu trúc cơ bản của CPU (3)

1. Đường đi của dữ liệu (2)

Các thành phần:

- Bộ phận số học và luận lý (ALU)
- Các thanh ghi
 - Tổng quát
 - Bộ đếm chương trình (PC)
 - Trạng thái (SR)
 - Đệm (TEMP)
 - Địa chỉ bộ nhớ (MAR)
 - Dữ liệu bộ nhớ (MDR)
 - MUX (Multiplexer): Bộ đa hợp.
- Các đường nối kết
 - Dữ liệu nguồn 1 (S1)
 - Dữ liệu nguồn 2 (S2)
 - Dữ liệu đích (Dest)



Hình III.1: Tổ chức của một xử lý điển hình
(Các đường không liên tục là các đường điều khiển)

2. Bộ điều khiển (1)

- **Nhiệm vụ**

- Tạo các tín hiệu điều khiển di chuyển số liệu.
- Điều khiển thực hiện các tác vụ của các bộ phận chức năng (ALU, R/W bộ nhớ).
- Tạo các tín hiệu điều khiển thực hiện các lệnh một cách tuần tự.

- **Phân loại:**

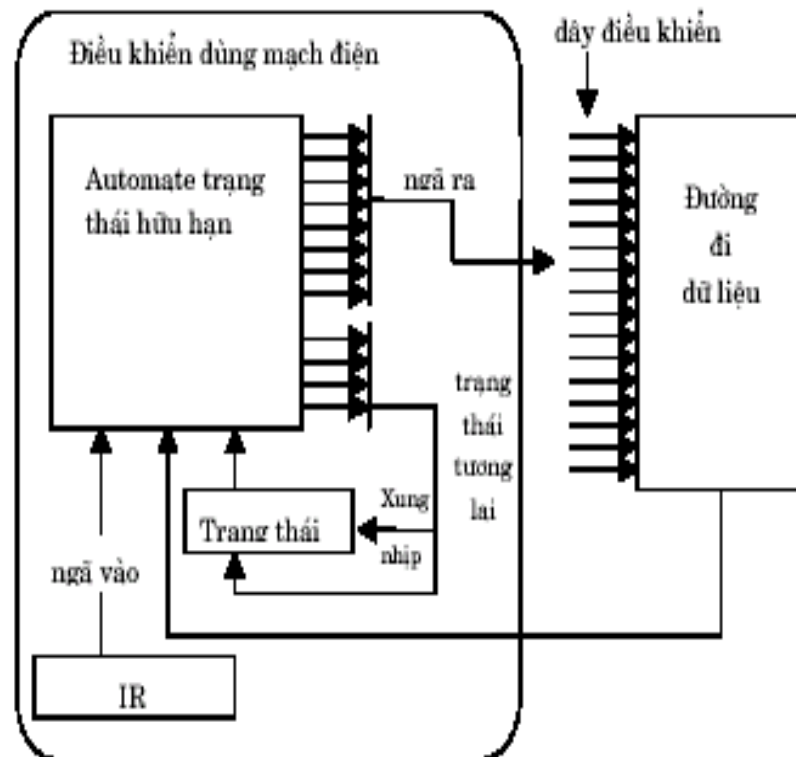
- Bộ điều khiển dùng mạch điện tử.
- Bộ điều khiển dùng vi chương trình.

4.1. Cấu trúc cơ bản của CPU (4)

2. Bộ điều khiển

❖ Bộ điều khiển mạch điện tử:

- Các đường điều khiển là các ngõ ra của một hoặc nhiều automat trạng thái hữu hạn.
- Các ngõ vào của automat gồm có thanh ghi lệnh, trạng thái hiện tại và những thông tin từ bộ đường đi số liệu.
- Ứng với các ngõ vào, automat sẽ cho các đường điều khiển phần đường đi số liệu (ngõ ra) và trạng thái tương lai.



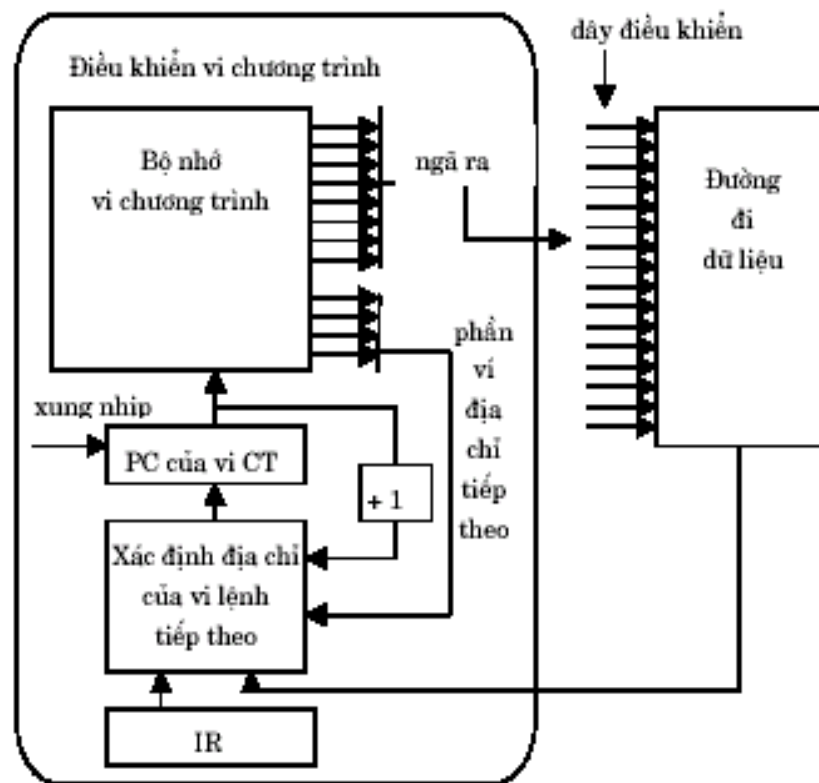
- ❖ **Nhận xét:** Kỹ thuật điều khiển dùng mạch điện tử đơn giản và hữu hiệu khi các lệnh có chiều dài cố định, có dạng thức đơn giản. Nó được dùng nhiều trong các bộ xử lý RISC.

4.1. Cấu trúc cơ bản của CPU (5)

2. Bộ điều khiển

❖ Bộ điều khiển vi chương trình:

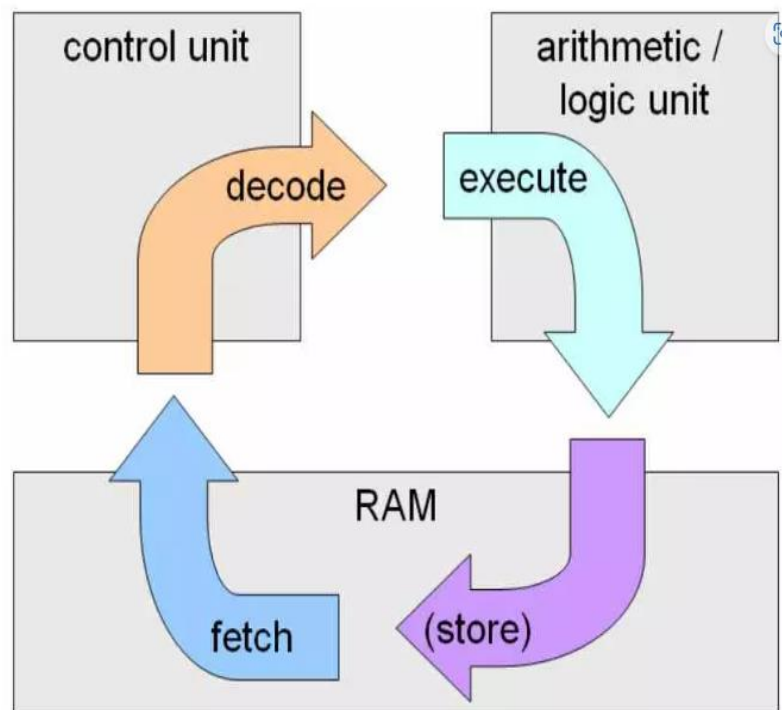
- Các đường điều khiển bộ đường đi dữ liệu ứng với các ngã ra của một vi lệnh trong bộ nhớ vi chương trình.
- Điều khiển các tác vụ của lệnh mã máy bằng cách thực hiện một chuỗi các vi lệnh.
- Một vi máy tính bên trong bộ điều khiển thực hiện từng lệnh của vi chương trình.
- Các tác vụ của lệnh mã máy tùy thuộc vào trạng thái của phần đường đi số liệu.



❖ **Nhận xét:** Bộ điều khiển vi chương trình được dùng trong các CPU CISC, người ta cài đặt một lệnh mã máy bằng một vi chương trình.

4.2. Hoạt động của CPU (1)

1. Chu kỳ lệnh (Instruction cycle) (1)

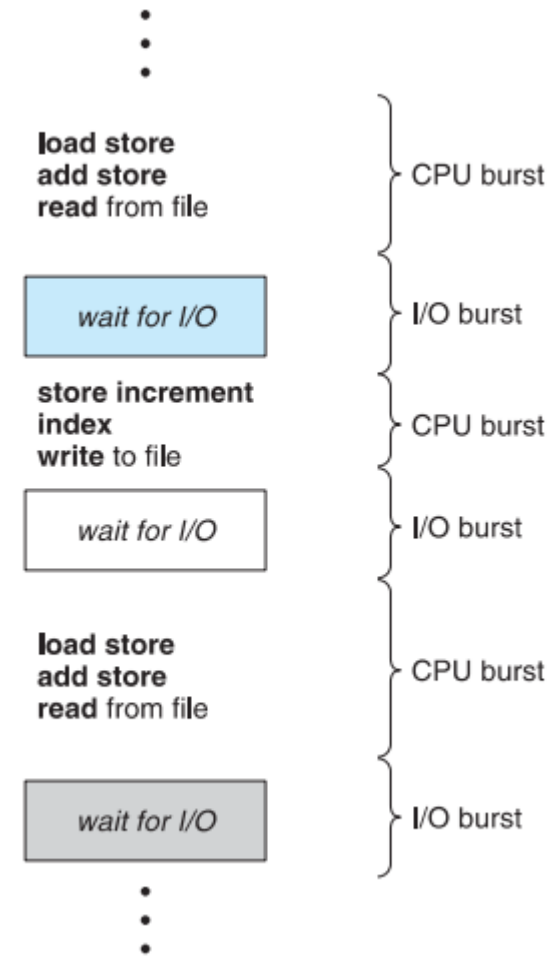


- **Chu kỳ lệnh** là chu trình mà CPU tuân theo để xử lý các lệnh. Nó bao gồm 3 giai đoạn chính: **giai đoạn tìm nạp** (fetch stage), **giai đoạn giải mã** (decode stage) và **giai đoạn thực thi** (execute stage).
- Tuy nhiên, để thấy rõ hơn quá trình thực thi 1 lệnh, người ta thường phân chia chu kỳ lệnh thành 5 giai đoạn:
 - 1) **Nạp lệnh**
 - 2) **Giải mã lệnh và đọc toán hạng**
 - 3) **Thi hành lệnh**
 - 4) **Thâm nhập bộ nhớ hoặc nhảy**
 - 5) **Lưu trữ kết quả**

4.2. Hoạt động của CPU (2)

1. Chu kỳ lệnh (Instruction cycle) (2)

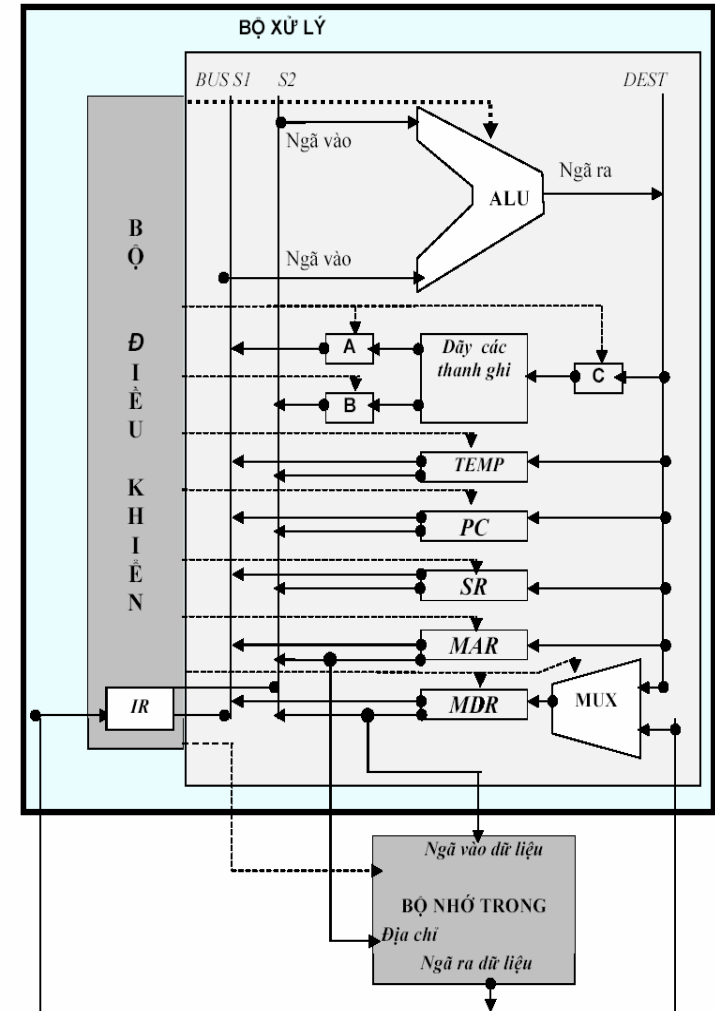
- **Cần phân biệt các khái niệm:**
 - Chu kỳ lệnh (Instruction cycle)
 - Chu kỳ CPU (CPU burst)
 - Chu kỳ I/O (I/O burst)
 - Chu kỳ xung đồng hồ (Clock cycle)
- **CPU burst** và **I/O burst** là hai khái niệm liên quan đến tiến trình (Process) thực hiện các chương trình. CPU burst là thời gian mà một tiến trình được sử dụng CPU (để thực hiện các lệnh tính toán) trước khi nó không còn sẵn sàng nữa. I/O burst là thời gian mà một tiến trình phải chờ đợi để thực hiện các hoạt động I/O.
- Một chu kỳ lệnh có nhiều xung clock, một chu kỳ CPU có nhiều lệnh, một chu kỳ I/O cũng có nhiều xung clock.



4.2. Hoạt động của CPU (2)

2. Diễn tiến thi hành lệnh mã máy

1. Nạp lệnh: $MAR \leftarrow PC$
 $IR \leftarrow M[MAR]$
2. Giải mã lệnh và đọc toán hạng:
 - $A \leftarrow Rs1/Tg\ DC$
 - $B \leftarrow Rs2/Độ\ dời$
 - $PC \leftarrow PC + 4$
3. Thi hành lệnh:
 - **Thâm nhập bộ nhớ:** $MAR \leftarrow \text{Địa chỉ } (A+B)$
 - Ghi: $MDR \leftarrow \text{Dữ liệu (Th Số liệu)}$
 - **Lệnh của ALU:**
 Ngã ra ALU $\leftarrow$ Kết quả phép tính
 - **Lệnh nhảy:**
 Ngã ra ALU $\leftarrow$ Địa chỉ (ALU tính).
4. Thâm nhập bộ nhớ / nhảy lần cuối:
 - **Thâm nhập bộ nhớ:**
 - Đọc: $MDR \leftarrow M[MAR]$
 - Ghi: $M[MAR] \leftarrow MDR$
 - **Nhảy:** If (điều kiện), $PC \leftarrow$ ngã ra ALU
5. Lưu trữ kết quả:
 - **Thâm nhập bộ nhớ:** - Đọc: $Rd \leftarrow MDR$
 - **Lệnh của ALU:** $Rd \leftarrow$ Ngã ra ALU



Hình III.1: Tổ chức của một xử lý điển hình
 (Các đường không liên tục là các đường điều khiển)

4.2. Hoạt động của CPU (3)

3. Kỹ thuật ống dẫn (Pipelining)

- **Khái niệm:** Kỹ thuật ống dẫn là một kỹ thuật làm cho các giai đoạn khác nhau của nhiều lệnh được thi hành cùng một lúc.
- **Các giai đoạn:**
 - Lấy lệnh (FO: Fetch Operation),
 - Giải mã (DE: Decode),
 - Thi hành (EX : Execute),
 - Tham nhập bộ nhớ (MEM : Memory access),
 - Lưu trữ kết quả (RS : Result Storing).

Lệnh	Chu kỳ xung nhịp								
	1	2	3	4	5	6	7	8	9
Lệnh i	FO	DE	EX	MEM	RS				
Lệnh i+1		FO	DE	EX	MEM	RS			
Lệnh i+2			FO	DE	EX	MEM	RS		
Lệnh i+3				FO	DE	EX	MEM	RS	
Lệnh i+4					FO	DE	EX	MEM	RS

Các giai đoạn khác nhau của nhiều lệnh được thi hành cùng một lúc.

3. Kỹ thuật ống dẫn (2)

❖ Các ràng buộc:

- Phải có một mạch điện để thi hành một giai đoạn của lệnh.
- Phải có nhiều thanh ghi.
- Giải mã các lệnh phải đơn giản
- Cần phải có các bộ làm tính ALU hữu hiệu.
- Cần phải có nhiều thanh ghi lệnh.
- Có nhiều thanh ghi bộ đếm chương trình

4.2. Hoạt động của CPU (5)

3. Kỹ thuật ống dẫn-Khó khăn trong kỹ thuật ống dẫn

- Khi thi hành lệnh trong một máy tính dùng kỹ thuật ống dẫn, có nhiều trường hợp làm cho kỹ thuật ống dẫn không thực hiện được hoặc thực hiện không hiệu quả:
 - Thiếu các mạch chức năng,
 - Một lệnh dùng kết quả của lệnh trước,
 - Một lệnh nhảy.

1. Khó khăn do cấu trúc

- Nguyên nhân: Thiếu bộ phận chức năng (ALU, PC, IR, ...) thoả mãn các ràng buộc của kỹ thuật ống dẫn.
- Khắc phục: Trang bị thêm các bộ phận chức năng cần thiết & hiệu quả

4.2. Hoạt động của CPU (6)

3. Kỹ thuật ống dẫn - Khó khăn trong kỹ thuật ống dẫn

2. Khó khăn do số liệu:

- Nguyên nhân: Lệnh kế sau có sử dụng kết quả của lệnh kế trước.

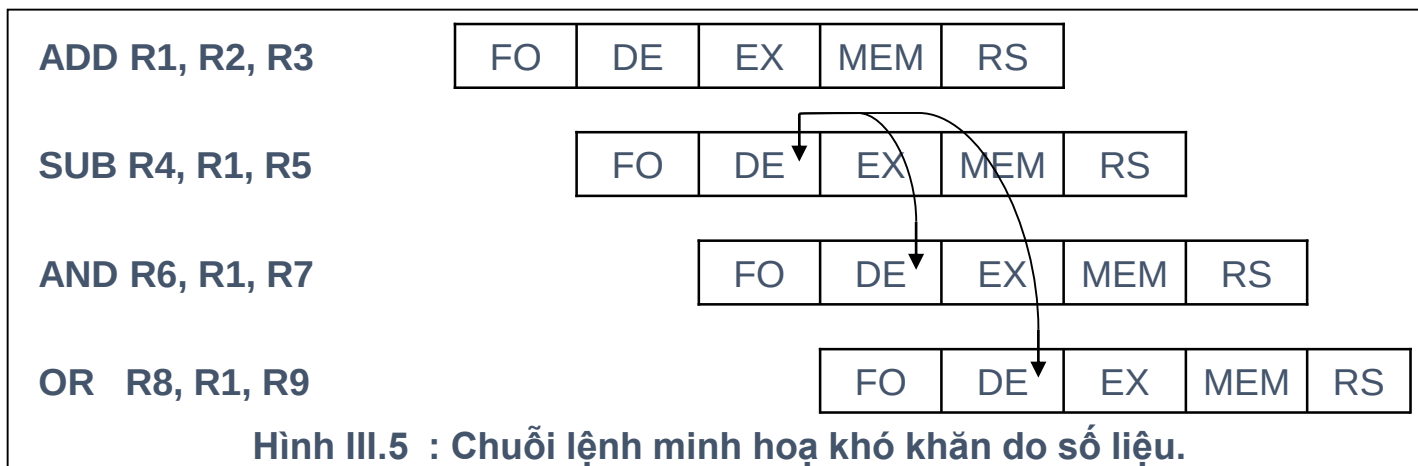
- Ví dụ:

1- ADD R1, R2, R3

2- SUB R4, R1, R5

3- AND R6, R1, R7

4- OR R8, R1, R9

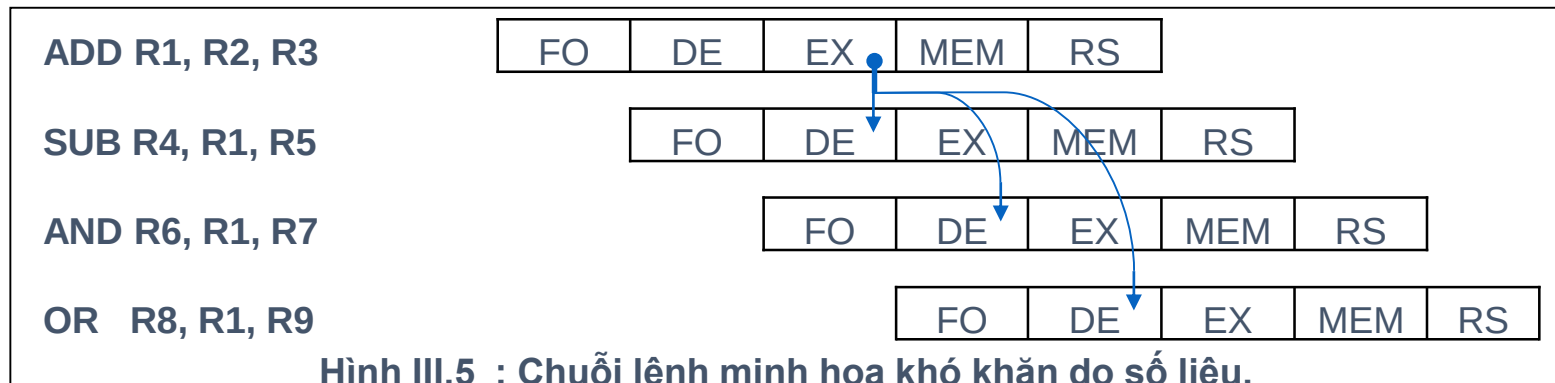
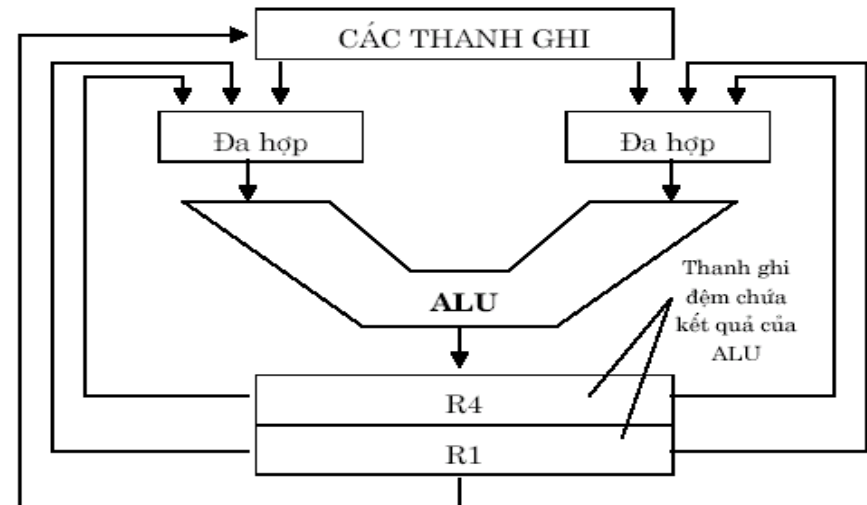


4.2. Hoạt động của CPU (7)

3. Kỹ thuật ống dẫn - Khó khăn trong kỹ thuật ống dẫn

2. Khó khăn do số liệu:

- Khắc phục:** Trang bị thêm bộ phận phần cứng đặc biệt để đưa kết quả ở ngõ ra ALU, trực tiếp vào một trong các thanh ghi ngõ vào.



4.2. Hoạt động của CPU (8)

3. Kỹ thuật ống dẫn - Khó khăn trong kỹ thuật ống dẫn

3. Khó khăn do điều khiển

- Nguyên nhân: Các lệnh nhảy làm thay đổi tính tuần tự khi thi hành các lệnh

Đối với lệnh nhảy không điều kiện:

- Tác vụ nhảy không thể biết trước giai đoạn giải mã. Nếu lệnh nhảy bắt đầu ở chu kỳ c thì lệnh nhảy tới bắt đầu ở chu kỳ $c+2$.
- Trong lệnh nhảy tương đối, ta phải cộng độ dời chứa trong thanh ghi IR vào thanh ghi PC. Việc tính địa chỉ được thực hiện vào giai đoạn DE với điều kiện phải có một mạch cộng riêng biệt.

Lệnh	Chu kỳ xung nhịp								
	1	2	3	4	5	6	7	8	9
Lệnh i	FO	*DE	EX	MEM	RS				
Lệnh $i+1$		FO	DE	EX	MEM	RS			
Lệnh $i+2$			*FO	DE	EX	MEM	RS		
Lệnh $i+3$				FO	DE	EX	MEM	RS	
Lệnh $i+4$					FO	DE	EX	MEM	RS

4.2. Hoạt động của CPU (8)

3. Kỹ thuật ống dẫn - Khó khăn trong kỹ thuật ống dẫn

3. Khó khăn do điều khiển

- **Nguyên nhân:** Các lệnh nhảy làm thay đổi tính tuần tự khi thi hành các lệnh

Đối với các lệnh nhảy có điều kiện:

- Phải tính toán điều kiện. Trong kiến trúc RISC, kết quả so sánh đặt vào thanh ghi trạng thái / tổng quát. Trong cả 2 trường hợp, đọc điều kiện tương đương với đọc thanh ghi.
- Trường hợp khó hơn có thể xảy ra trong lệnh nhảy có điều kiện:
 - Điều kiện có được khi so sánh 2 thanh ghi và chỉ thực hiện lệnh nhảy khi kết quả so sánh là đúng.
 - Tính toán trên các đại lượng logic không thể thực hiện trong phân nửa chu kỳ và như thế phải kéo dài thời gian thực hiện lệnh nhảy.

⇒ **Người ta thường tránh các trường hợp này để không làm giảm mức hữu hiệu của máy tính.**

4.2. Hoạt động của CPU (9)

3. Kỹ thuật ống dẫn - Khó khăn trong kỹ thuật ống dẫn

❖ Khắc phục:

- **Cách thứ nhất:** Đóng băng kỹ thuật ống dẫn trong một chu kỳ, nghĩa là ngưng thi hành lệnh thứ $i+1$ đang làm nếu lệnh thứ i là lệnh nhảy. Ta mất trắng một chu kỳ cho mỗi lệnh nhảy.
- **Cách thứ hai:** Thi hành lệnh sau lệnh nhảy (lưu ý rằng hiệu quả của một lệnh nhảy bị chậm mất một lệnh). Vậy lệnh theo sau lệnh nhảy được thực hiện trước khi lệnh mà chương trình phải nhảy tới được thực hiện. Chương trình dịch hay người lập trình có nhiệm vụ xen vào một lệnh hữu ích sau lệnh nhảy.

Lưu ý: Trong trường hợp nhảy có điều kiện, việc nhảy có thể được thực hiện hay không. Lệnh hữu ích đặt sau lệnh nhảy không làm sai chương trình dù điều kiện nhảy đúng hay sai.

Ví dụ: Bộ xử lý SPARC có những lệnh nhảy với hủy bỏ. Các lệnh này cho phép thi hành lệnh sau lệnh nhảy nếu điều kiện nhảy **sai** và hủy bỏ thực hiện lệnh đó nếu điều kiện nhảy **đúng**.

4.3. Tập lệnh của CPU (1)

- Tập lệnh là tập hợp các lệnh mã máy của bộ xử lý.
- Mục tiêu của phần này là dùng các ví dụ trích từ các kiến trúc phần mềm được dùng nhiều nhất, để cho thấy các kỹ thuật ở mức ngôn ngữ máy dùng để thi hành các cấu trúc trong ngôn ngữ cấp cao.
- Để minh họa ta dùng cú pháp lệnh trong hợp ngữ sau đây :
 - <Từ gợi nhớ mã lệnh> <Thanh ghi đích>, <Thanh ghi nguồn 1>, <Thanh ghi nguồn 2>
 - Từ gợi nhớ mã lệnh mô tả ngắn gọn tác vụ phải thi hành trên các thanh ghi nguồn, kết quả được lưu giữ trong thanh ghi đích.
 - Mỗi lệnh của ngôn ngữ cấp cao được xây dựng bằng một lệnh mã máy hoặc một chuỗi nhiều lệnh mã máy.
- ❖ **Lệnh nhảy (GOTO)** được thực hiện bằng các lệnh hợp ngữ về nhảy (JUMP) hoặc lệnh hợp ngữ về vòng. Chúng ta phân biệt:
 - **Lệnh nhảy đến địa chỉ tuyệt đối:** làm cho bộ đếm chương trình được nạp vào địa chỉ tuyệt đối phải nhảy đến ($PC \leftarrow$ địa chỉ tuyệt đối phải nhảy tới),
 - **Lệnh nhảy theo độ dời (vòng):** theo đó ta chỉ cần cộng thêm một độ dời vào bộ đếm chương trình ($PC \leftarrow PC + \text{độ dời}$).

4.3. Tập lệnh của CPU (2)

❖ **Gán trị:** Việc gán trị được thực hiện nhờ một số lệnh mã máy (gồm cả gán trị cho biểu thức số học và logic). Cho kiến trúc RISC, ta có:

- **Lệnh bộ nhớ:**

LOAD Ri, (địa chỉ) M[địa chỉ] → Ri

STORE Ri, (địa chỉ) Ri → M[địa chỉ]

- **Lệnh tính toán số nguyên** trên nội dung của hai thanh ghi Ri, Rj và xếp kết quả vào trong Rk:

ADD (cộng)

ADDD (cộng số có dấu chấm động, chính xác kép)

SUB (trừ)

SUBD (trừ số có dấu chấm động, chính xác kép)

MUL (nhân)

DIV (chia)

- **Lệnh logic** thực hiện phép tính logic cho từng bit một:

AND (lệnh VÀ)

OR (lệnh HOẶC)

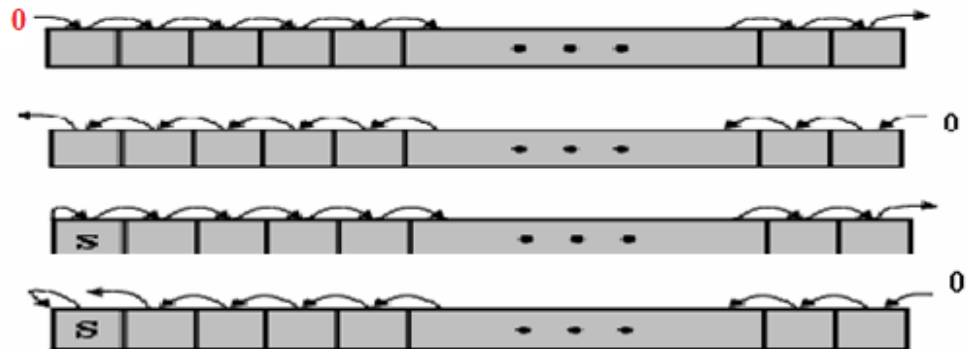
XOR (lệnh HOẶC LOẠI)

NEG (lệnh lấy số bù 1)

4.3. Tập lệnh của CPU (3)

❖ Các lệnh dịch chuyển số học hoặc logic (SHIFT):

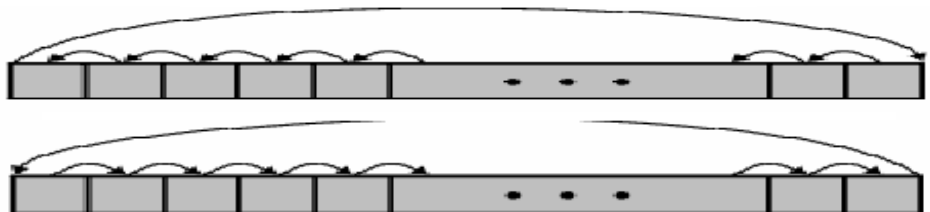
- Dịch phải logic
- Dịch trái logic
- Dịch phải số học
- Dịch trái số học



❖ Quay vòng (ROTATE):

- Có hoặc không có số giữ ở ngã vào,
- Sang phải hoặc sang trái.
- Thực hiện trên một thanh ghi và kết quả lưu trong thanh ghi khác.
- Số lần dịch chuyển thường được xác định trong thanh ghi thứ ba.

- Quay trái
- Quay phải



4.3. Tập lệnh của CPU (5)

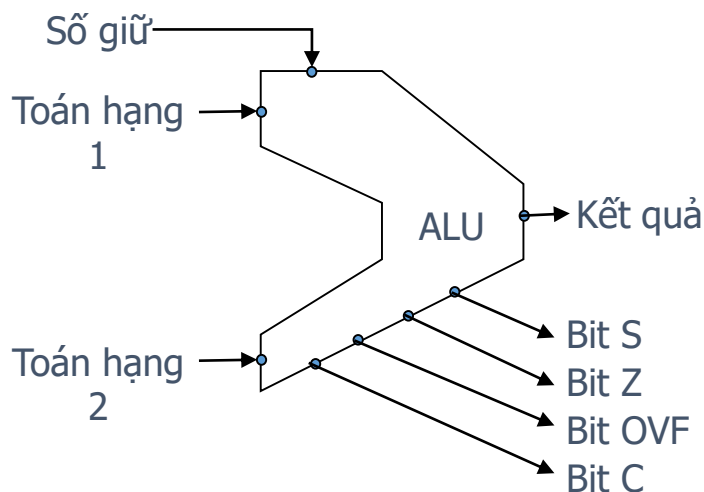
❖ Lệnh có điều kiện:

Lệnh có điều kiện có dạng :

Nếu <điều kiện> thì <chuỗi lệnh 1> nếu không <chuỗi lệnh 2>
(IF <condition> THEN <instructions> ELSE <instructions>)

Lệnh này phải ghi nhớ điều kiện và nhảy vòng nếu điều kiện thỏa.

a) Ghi nhớ điều kiện: Bộ làm tính ALU cung cấp kết quả ở ngã ra tùy theo các ngã vào và phép tính cần làm. Nó cũng cho một số thông tin khác về kết quả dưới dạng các bit trạng thái.



Trong các bit trạng thái ta có:

- **Bit dấu S** (1 nếu kết quả âm),
- **Bit trắc nghiệm zero Z** (1 nếu kết quả bằng 0).
- **Bit tràn OVF** (overflow) (1 nếu không đủ khả năng lưu kết quả).
- **Bit số giữ C** (carry) (1 nếu số giữ ở ngã ra là 1)

Các bit trên thường được gọi là bit mã điều kiện.

4.3. Tập lệnh của CPU (5)

Có hai kỹ thuật cơ bản để ghi nhớ các bit trạng thái

- Cách thứ nhất là ghi các bit trạng thái trong một **thanh ghi đa dụng**.

Ví dụ lệnh: CMP Rk, Ri, Rj Thực hiện phép tính $R_i - R_j$ không ghi kết quả phép trừ, ghi các bit trạng thái vào thanh ghi Rk.

- **Điểm lợi:** Lưu trữ nhiều trạng thái để dùng về sau.
- **Điểm bất lợi:** Phải dùng một thanh ghi để ghi lại trạng thái.
- Cách thứ hai là để các bit trạng thái vào một **thanh ghi đặc biệt**.
Lưu giữ nội dung thanh ghi này được giải quyết bằng nhiều cách:
 - Trong kiến trúc SPARC chỉ có một số giới hạn lệnh được phép thay đổi thanh ghi trạng thái.

Ví dụ:

Lệnh ADDCC phép cộng và làm thay đổi thanh ghi trạng thái.

Lệnh SUBCC phép trừ và làm thay đổi thanh ghi trạng thái.

- Trong kiến trúc PowerPC thanh ghi trạng thái được phân thành 8 trường 4 bit, vậy là đã phân thành 8 thanh ghi trạng thái con.

4.3. Tập lệnh của CPU (6)

b) Nhảy vòng:

- Lệnh nhảy / nhảy vòng có điều kiện (thực hiện khi điều kiện thoả).

Ví dụ: Giả sử trạng thái sau khi bộ xử lý thi hành một tác vụ, được lưu trữ trong thanh ghi, và bộ xử lý thi hành các lệnh sau :

- | | |
|-------------------|---|
| 1. CMP R4, R1, R2 | : So sánh R1 và R2 và lưu giữ trạng thái trong R4 |
| 2. BGT R4, +2 | : Nhảy bỏ 2 lệnh nếu $R1 > R2$ |
| 3. ADD R3, R0, R2 | : R0 có giá trị 0. Chuyển nội dung của R2 vào R3 |
| 4. BRA +1 | : Nhảy bỏ 1 lệnh |
| 5. ADD R3, R0, R1 | : Chuyển nội dung R1 vào R3 |
| 6. Lệnh kế | |

- Nếu $R1 > R2$ thì chuỗi lệnh được thi hành là 1, 2, 5, 6 được thi hành,

- Nếu không thì chuỗi lệnh 1, 2, 3, 4, 6 được thi hành.

- Các lệnh thực hiện công việc: Nếu $R1 > R2$ thì $R3 = R1$ nếu không $R3 = R2$

- **Lưu ý:** Các lệnh nhảy làm giảm tốc độ thi hành lệnh, trong các CPU hiện đại dùng kỹ thuật ống dẫn. Trong một vài bộ xử lý người ta dùng lệnh di chuyển có điều kiện để tránh dùng lệnh nhảy trong một vài trường hợp.

Ví dụ được viết lại :

- | | |
|-------------------|--|
| 1. CMP R4, R1, R2 | : So sánh R1 và R2 và để các bit trạng thái trong R4. |
| 2. ADD R3, R0, R2 | : Di chuyển R2 vào R3 |
| 3. MGT R4, R3, R1 | : (MGT: Move if greater than). Nếu $R1 > R2 \Rightarrow R1 \rightarrow R3$ |

4.3. Tập lệnh của CPU (7)

❖ **Vòng lặp:** Lệnh vòng lặp được thực hiện nhờ lệnh nhảy có điều kiện. Quản lý số lần lặp lại bằng một bộ đếm vòng lặp và kiểm tra bộ đếm sau mỗi vòng lặp để xem đã đủ số vòng cần thực hiện hay chưa?

Bộ xử lý PowerPC có một lệnh quản lý vòng lặp:

BNCT Ri, độ dời (Thanh ghi Ri chứa số lần lặp lại).

Lệnh làm công việc sau: $Ri := Ri - 1$

Nếu $Ri=0$, $PC:=PC+\text{độ dời}$.

Nếu không thì tiếp tục thi hành lệnh lặp.

4.3. Tập lệnh của CPU (8)

❖ Thâm nhập bộ nhớ ngăn xếp:

Ngăn xếp là một tổ chức bộ nhớ sao cho ta chỉ có thể đọc một từ ở đỉnh ngăn xếp hoặc viết một từ vào đỉnh ngăn xếp. Địa chỉ của đỉnh ngăn xếp chứa trong một thanh ghi đặc biệt gọi là con trỏ ngăn xếp SP (stack pointer).

- Lệnh PUSH: $SP := SP + d$; Tăng thanh ghi SP một từ.
 $M(SP) := Ri$; Lưu Ri vào ngăn xếp.
- Lệnh POP: $Ri := M(SP)$; Đọc ngăn xếp $\rightarrow Ri$.
 $SP := SP - d$; Giảm thanh ghi SP một từ.

➤ d là 1 từ máy tính, mặc định $d = 4$

Ví dụ: Trong các bộ xử lý RISC, thanh ghi R30 là con trỏ ngăn xếp.
Viết vào ngăn xếp:

ADDI	R30, R30, 4	; Tăng SP lên 4 bytes vì một từ dài 32 bit
STORE	$Ri, (R30)$	; Viết Ri vào đỉnh ngăn xếp

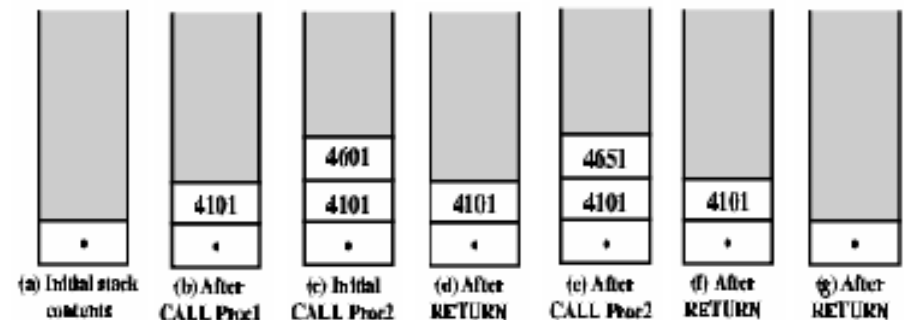
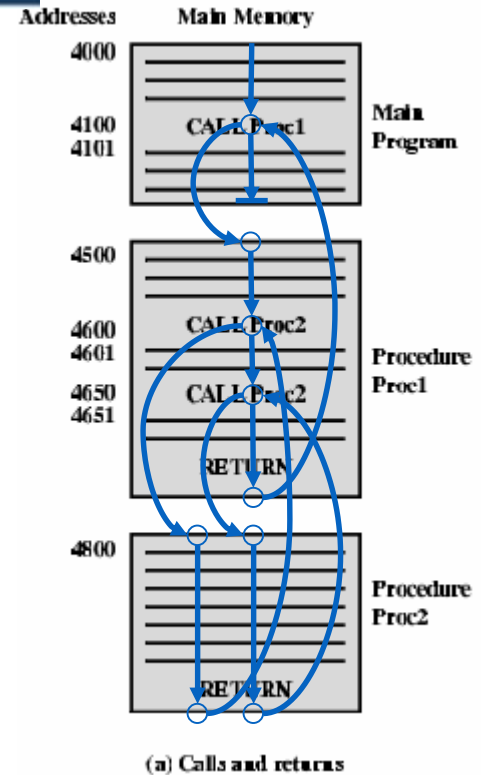
Lấy ra khỏi ngăn xếp:

LOAD	$Ri, (R30)$	; Lấy số liệu ở đỉnh ngăn xếp nạp vào Ri
SUBI	R30, R30, 4	; Giảm SP một từ (4 bytes).

4.3. Tập lệnh của CPU (9)

• Các thủ tục:

- Các thủ tục được gọi từ bất cứ nơi nào của chương trình nhờ lệnh gọi thủ tục CALL.
- Để khi chấm dứt việc thi hành thủ tục, chương trình gọi được tiếp tục bình thường, ta cần lưu địa chỉ trở về. Khi chấm dứt thủ tục, lệnh RET nạp địa chỉ trở về vào PC.
- Trong kiến trúc CISC, địa chỉ trở về được giữ ở ngăn xếp.



4.3. Tập lệnh của CPU (10)

Lưu ý:

- Thanh ghi lưu địa chỉ trở về chỉ áp dụng cho thủ tục cuối.
- Các thủ tục có thể gọi thủ tục khác, ta dùng bộ nhớ ngăn xếp

Gọi thủ tục 1: JMPL Ri

R31 := PC ; lưu địa chỉ trở về trong R31
PC := Ri ; nhảy tới địa chỉ thủ tục (Ri)

Gọi thủ tục 2:

ADDI R30, R30,4 ; R30 là con trỏ ngăn xếp.
STORE R31, (R30) ; Lưu giữ địa chỉ trở về chương trình chính.
JMPL Rj ; Gọi thủ tục 2
R31 := PC ; Lưu địa chỉ trở về thủ tục 1
PC := Rj ; Gọi thủ tục.

Trở về thủ tục 1: JMP R31

PC:=R31

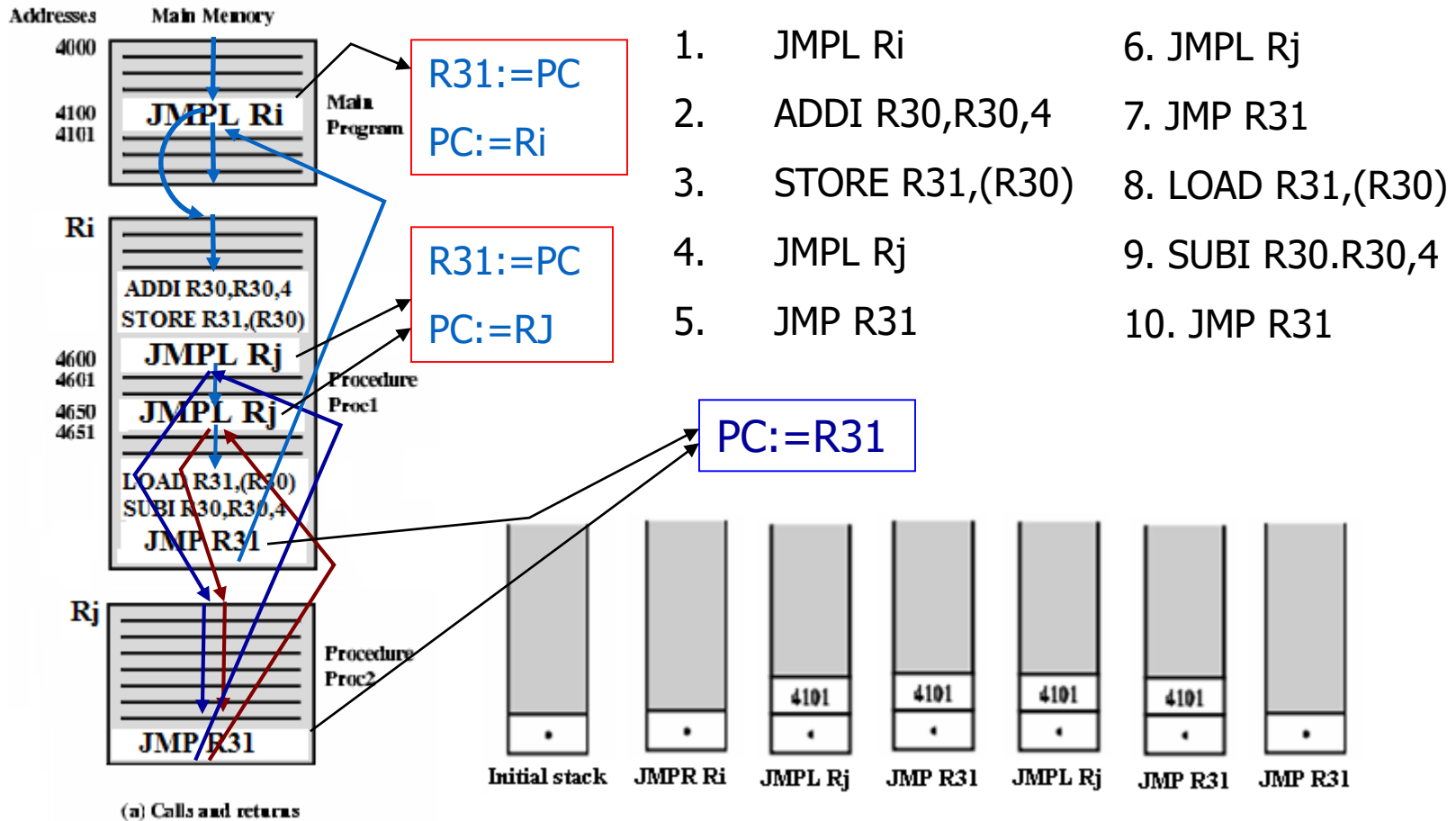
Trở về chương trình chính:

LOAD R31, (R30) ; Phục hồi địa chỉ trở về
SUBI R30, R30,4 ; Cập nhật con trỏ ngăn xếp
JMP R31 ; Trở về chương trình gọi
PC:=R31

Truyền tham số từ thủ tục gọi đến thủ tục bị gọi có thể thực hiện bằng cách dùng các thanh ghi (ít tham số) hoặc dùng ngăn xếp.

4.3. Tập lệnh của CPU (10)

- Trở lại ví dụ như trong KT CISC



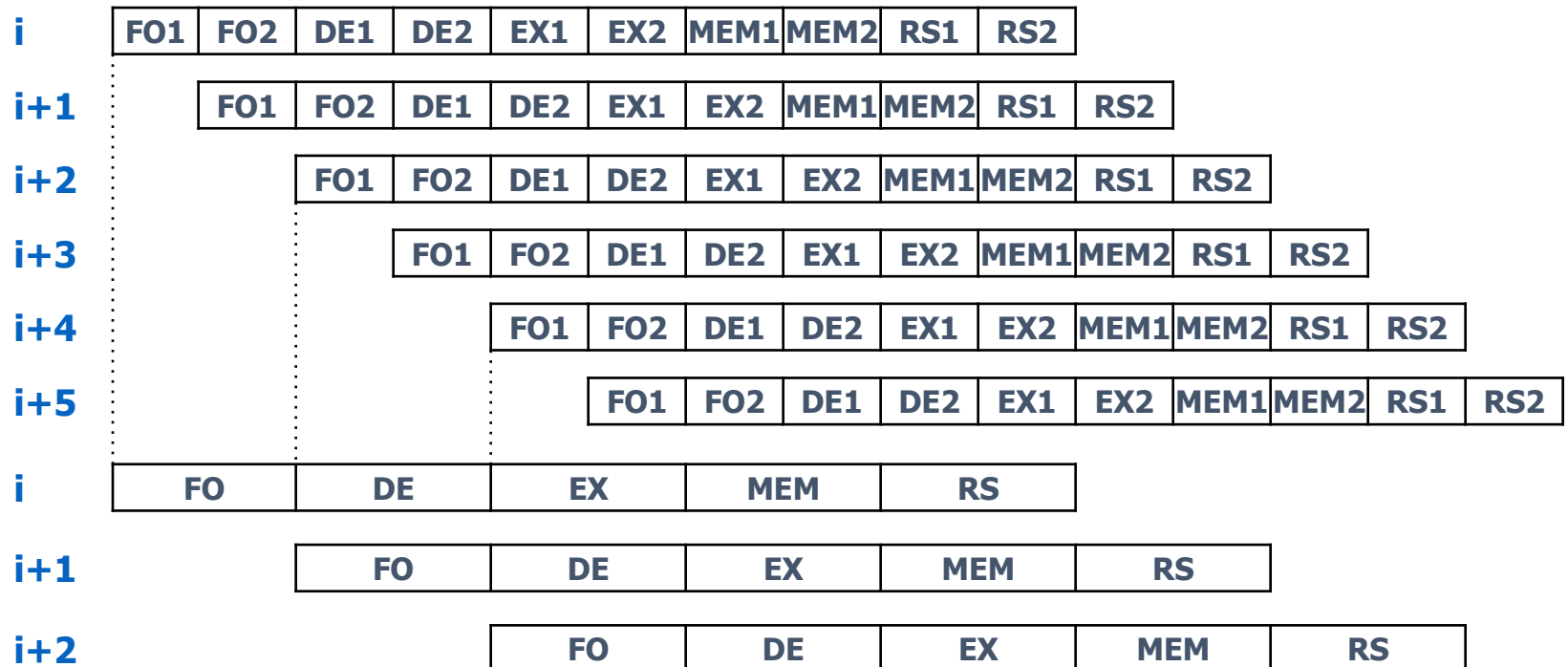
4.3. Tập lệnh của CPU (11)

Lưu ý:

- Thanh ghi lưu địa chỉ trở về chỉ áp dụng cho thủ tục cuối.
- Các thủ tục có thể gọi thủ tục khác, ta dùng bộ nhớ ngăn xếp
 - Gọi thủ tục 1: JMPL Ri
 - $R31 := PC$; lưu địa chỉ trở về trong R31
 - $PC := Ri$; nhảy tới địa chỉ thủ tục (Ri)
 - Gọi thủ tục 2:
 - ADDI R30, R30,4** ; R30 là con trỏ ngăn xếp.
 - STORE R31, (R30)** ; Lưu giữ địa chỉ trở về chương trình chính.
 - JMPL Rj** ; Gọi thủ tục 2
 - $R31 := PC$; Lưu địa chỉ trở về thủ tục 1
 - $PC := Rj$; Gọi thủ tục.

1. Siêu ống dẫn (1)

- Khái niệm:** Người ta có kỹ thuật siêu ống dẫn bậc n bằng cách chia các giai đoạn của kỹ thuật ống dẫn đơn giản (mỗi giai đoạn được thực hiện trong thời gian T_c), thành n giai đoạn con thực hiện trong thời gian T_c/n .



1. Siêu ống dẫn (2)

- **Lợi điểm:**

- Độ hữu hiệu của kỹ thuật siêu ống dẫn tương đương với việc thi hành $5n$ lệnh trong mỗi chu kỳ T_c (thời gian thực hiện 1 lệnh không dùng KT ống dẫn)
- Ta thấy trong một chu kỳ T_c , máy dùng kỹ thuật siêu ống dẫn bậc 2 sẽ làm 2×5 lệnh thay vì làm 5 lệnh trong máy dùng kỹ thuật ống dẫn bình thường.
- Trong máy tính siêu ống dẫn, tốc độ thực hiện lệnh tương đương với việc thực hiện một lệnh trong khoảng thời gian $T_c/5n$.

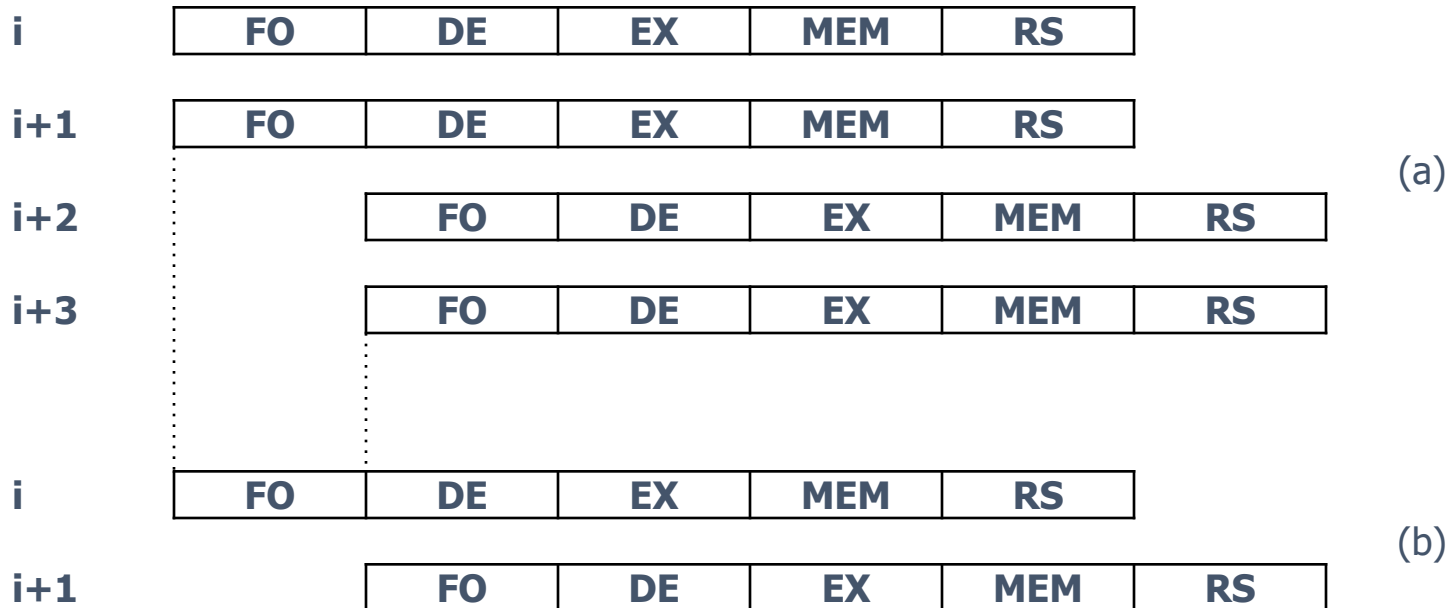
- **Bất lợi:**

- Bất lợi của siêu ống dẫn là thời gian thực hiện một giai đoạn con ngắn $T_c/5n$ và việc trì hoãn trong thi hành lệnh nhảy lớn.
- Nếu lệnh thứ i là một lệnh nhảy tương đối thì lệnh này được giải mã trong giai đoạn DE, địa chỉ nhảy đến được tính vào giai đoạn EX, lệnh phải nhảy tới là lệnh thứ $i+4$, vậy có trì trễ 3 lệnh thay vì 1 lệnh trong kỹ thuật ống dẫn bình thường.

2. Siêu vô hướng (1)

- Khái niệm:** Máy tính siêu vô hướng bậc n có thể thực hiện đồng thời n lệnh trong một chu kỳ xung nhịp T_c .

Ví dụ: Sự vận hành của một máy tính siêu vô hướng bậc 2 so với một máy tính dùng kỹ thuật ống dẫn.



Hình III.8 : Siêu vô hướng (a) so với kỹ thuật ống dẫn (b).



4.4. Kiến trúc của các bộ xử lý tiên tiến (4)

2. Siêu vô hướng (2)

❖ Nhận xét:

- Trong một máy tính siêu vô hướng phần cứng phải quản lý việc đọc và thi hành đồng thời nhiều lệnh. Vậy nó phải có khả năng quản lý các quan hệ giữa số liệu với nhau.
- Cần chọn các lệnh có khả năng được thi hành cùng một lúc.

❖ Ví dụ:

- Những bộ xử lý đầu tiên đưa ra thị trường dùng kỹ thuật này là Intel i860 và IBM RS/6000. Các bộ xử lý này có khả năng thực hiện song song nhiều tác vụ trên số nguyên và trên số lẻ.
- Năm 1992, người ta thấy xuất hiện các bộ xử lý có nhiều bộ thực hiện tác vụ độc lập với nhau (nhiều ALU, bộ tính toán số lẻ, nạp dữ liệu, lưu dữ liệu, nhảy), có thể thực hiện song song nhiều lệnh. Số lệnh có thể được thi hành song song càng nhiều thì phần cứng thực hiện việc này càng phức tạp.



4.4. Kiến trúc của các bộ xử lý tiên tiến (5)

2. Siêu vô hướng (3)

- **Kỹ thuật siêu luồng (HT: Hyper Threading):**
 - Người ta tìm cách tận dụng các tài nguyên dư thừa của máy tính sử dụng kỹ thuật siêu vô hướng.
 - Giả lập một bộ xử lý vật lý dùng kỹ thuật siêu vô hướng thành 2 bộ xử lý logic để tăng hiệu quả thực thi các chương trình.
 - Tăng hiệu quả thực hiện các chương trình:
 - Đa chương trình mức độ tăng gần 30%
 - Đơn chương trình mức độ tăng đến 18%

2. Siêu vô hướng (4)

- **Kỹ thuật siêu luồng**

- Trong hệ thống cổ điển, đơn vị cơ bản sử dụng CPU là quá trình. Mỗi quá trình có 1 bộ các thanh ghi PC, SR, ngăn xếp và không gian địa chỉ riêng (Address Space).
- Trong hệ thống dùng kỹ thuật siêu luồng, đơn vị sử dụng CPU cơ bản là luồng. Một quá trình bao gồm một không gian địa chỉ và nhiều luồng điều khiển. Mỗi luồng có 1 bộ thanh ghi PC, SR, và ngăn xếp riêng, nhưng tất cả các luồng của một quá trình cùng chia sẻ một không gian địa chỉ. Nhờ đó các luồng có thể sử dụng các biến toàn cục, chia sẻ các tài nguyên như tập tin, ...
- Cách thức luồng chia sẻ CPU cũng giống như cách thức của một quá trình. Một luồng cũng có các trạng thái: đang chạy (running), sẵn sàng (ready), nghẽn (blocked) và kết thúc (dead).

3. Máy tính có lệnh thật dài (1)

❖ Nhận xét:

- Máy tính siêu vô hướng có thể thực hiện 2 hoặc 3 lệnh trong mỗi chu kỳ xung nhịp.
- Do kỹ thuật ống dẫn đòi hỏi các lệnh phải phụ thuộc vào nhau nên rất khó thực hiện nhiều lệnh trong một chu kỳ.
- Như vậy, thay vì thực hiện nhiều lệnh trong một chu kỳ, người ta tìm cách đưa vào nhiều lệnh (tác vụ) trong một từ lệnh dài.

❖ Đặc điểm:

- Một lệnh thật dài (VLIW: Very Long Instruction Word) có thể chứa 2 tác vụ tính toán số nguyên, 2 tác vụ tính toán số lẻ, 2 tác vụ thâm nhập bộ nhớ và một lệnh nhảy.
- Một lệnh như vậy được chia thành trường, mỗi trường có từ 16 đến 24 bit, chiều dài của lệnh VLIW từ 112 đến 168 bit.
- Có nhiều kỹ thuật tạo ra một lệnh VLIW trong đó tất cả các trường đều được dùng.
- Giá thành và độ phức tạp của một máy tính có lệnh thật dài tăng lên rất nhiều nếu tăng số trường trong một lệnh VLIW.

4. Máy tính vector (1)

❖ **Khái niệm:** Một máy tính vector bao gồm:

- Một bộ tính toán vô hướng bình thường dùng kỹ thuật ống dẫn
- Một bộ làm tính vector thực hiện các phép tính vector.
- Đa số các máy tính vector cho phép làm các phép tính trên:
 - Vector số nguyên,
 - Vector số lẻ,
 - Vector số logic (số Bool).

❖ **Các kiểu kiến trúc máy tính vector:**

- **Kiểu vector ô nhớ - ô nhớ:** Trong máy tính loại vector bộ nhớ - bộ nhớ, các phép tính vector được thực hiện trong bộ nhớ

Ví dụ: Các máy tính CDC thuộc loại này

- **Kiểu thanh ghi vector:** Có một bộ nhiều thanh ghi vector và những tác vụ vector được thực hiện trên các thanh ghi này.

Ví dụ: Các siêu máy tính: CRAY-1, CRAY-2, X-MP, Y-MP, NEC SX/2, Hitachi S820. Máy CRAY-2 (1995) có 8 thanh ghi vector, mỗi thanh ghi chứa 64 vector, mỗi vector có chiều dài 64 bit.

5. Máy tính song song (1)

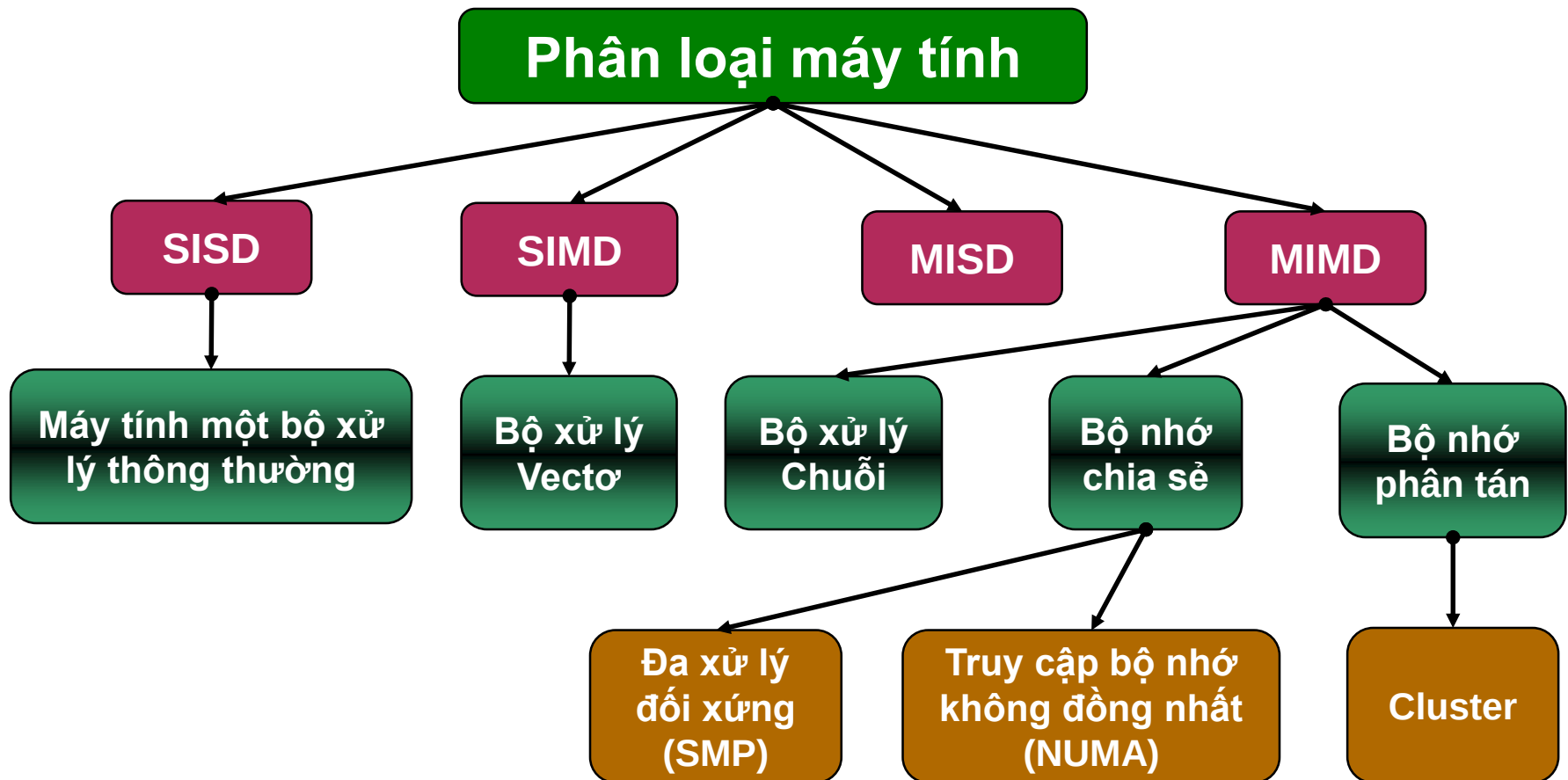
❖ Nhận xét:

- Trong các máy tính siêu ống dẫn, siêu vô hướng, máy tính vector, máy tính VLIW, người ta đã dùng tính thực hiện song song các lệnh ở các mức độ khác nhau để làm tăng hiệu quả.
- Giới hạn về khả năng tính toán của loại máy trên khiến người ta nghĩ tới giải pháp song song theo đó người ta tăng cường hiệu quả của máy tính bằng cách tăng số lượng bộ xử lý.

❖ Phân loại: Năm 1966, Flynn đã xếp các máy tính vào 4 loại :

- **SISD** (Single Instructions stream, Single Data stream): Máy tính một dòng lệnh, một dòng dữ liệu
- **SIMD** (Single Instructions stream, Multiple Data stream): Máy tính một dòng lệnh, nhiều dòng dữ liệu
- **MISD** (Multiple Instructions stream, Single Data stream): Máy tính nhiều dòng lệnh, một dòng dữ liệu
- **MIMD** (Multiple Instruction stream, Multiple Data stream): Máy tính nhiều dòng lệnh, nhiều dòng dữ liệu

5. Máy tính song song (2)



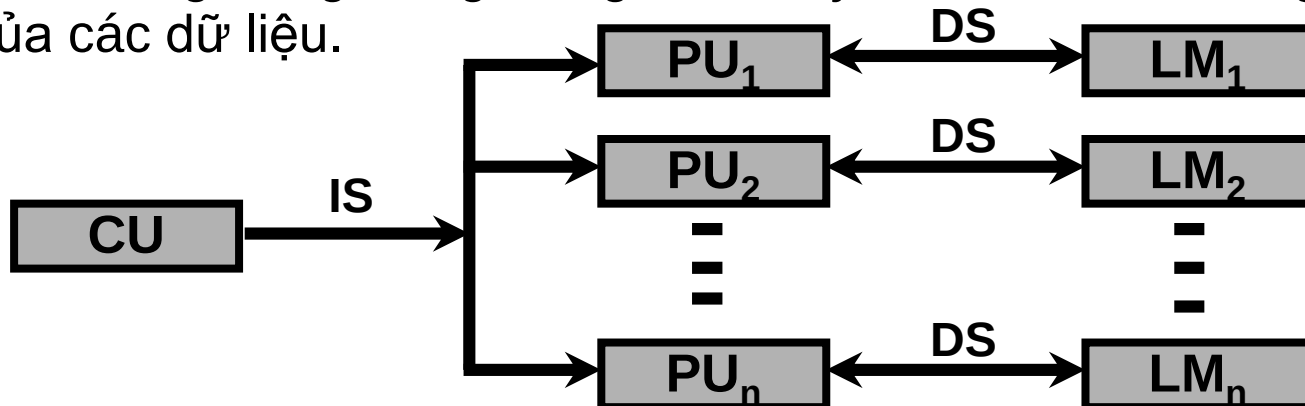
5. Máy tính song song (3)

SISD: Máy tính một dòng lệnh, một dòng dữ liệu là máy tính với 1 bộ xử lý mà ta đã nghiên cứu.



SIMD: Có một số lớn các bộ xử lý giống nhau, cùng thực hiện một lệnh giống nhau để xử lý nhiều dòng dữ liệu khác nhau.

- Mỗi bộ xử lý có bộ nhớ dữ liệu riêng, nhưng chỉ có một bộ nhớ lệnh và một bộ điều khiển, bộ này đọc và thi hành các lệnh.
- Tính song song dùng trong các máy SIMD là tính song song của các dữ liệu.



4.4. Kiến trúc của các bộ xử lý tiên tiến (12)

5. Máy tính song song (4)

MISD: Máy tính nhiều dòng lệnh, một dòng dữ liệu chưa chế tạo.

MIMD: Máy tính nhiều dòng lệnh, nhiều dòng dữ liệu là một kiến trúc đương nhiên cho các máy nhiều bộ xử lý dùng trong các ứng dụng thông thường.

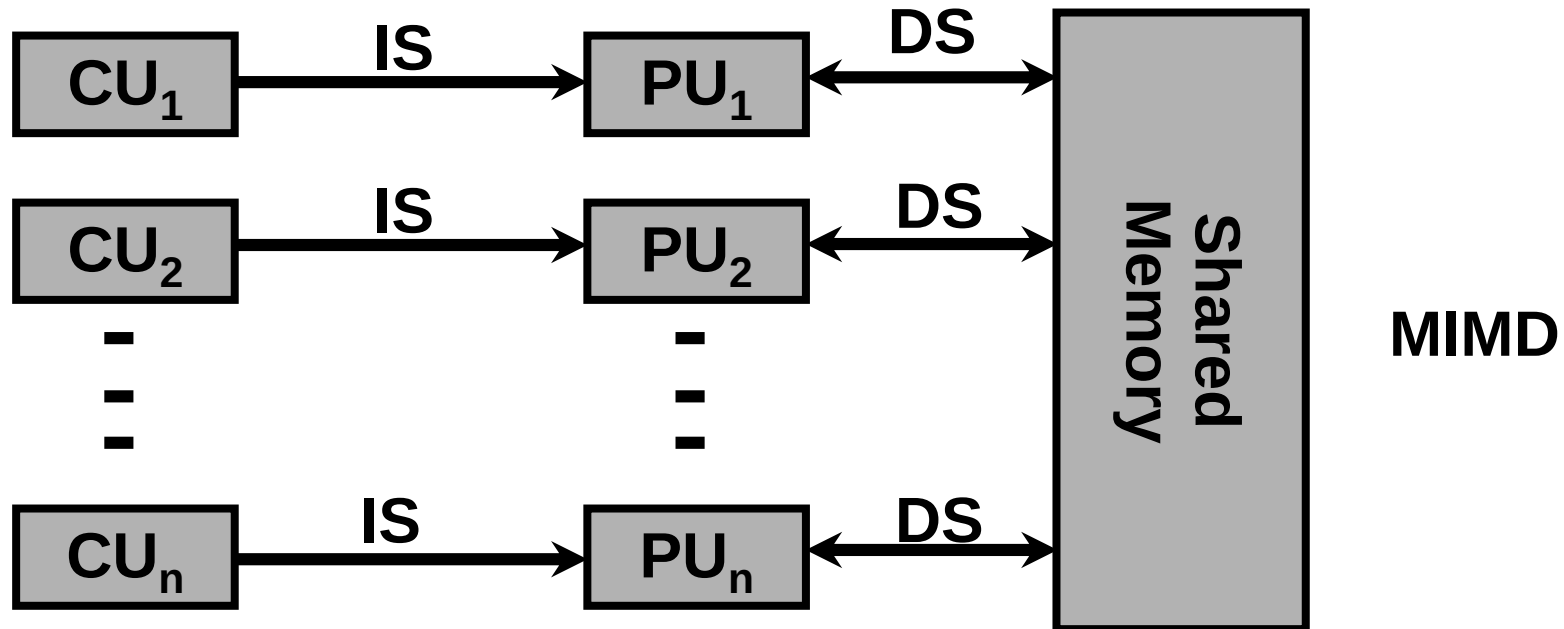
Người ta thường phân loại các máy MIMD theo tổ chức bộ nhớ:

- SMP (Symmetric Multiprocessors): Hệ thống đa xử lý bộ nhớ chia sẻ, thời gian truy cập bộ nhớ giống nhau cho các bộ xử lý.
- NUMA (NonUniform Memory Access): Hệ thống đa xử lý bộ nhớ chia sẻ, thời gian truy cập các vùng nhớ khác nhau thì khác nhau.
- CC-NUMA (Cache-Coherent NUMA): Hệ thống NUMA với sự liên kết và duy trì hệ thống cache trên các bộ xử lý riêng lẻ.
- Cluster: Toàn bộ các máy tính riêng lẻ được kết nối với nhau thực hiện các công việc như một máy tính hợp nhất có chung các tài nguyên. Mỗi máy tính được xem như là một bộ phận của Cluster.

5. Máy tính song song (5)

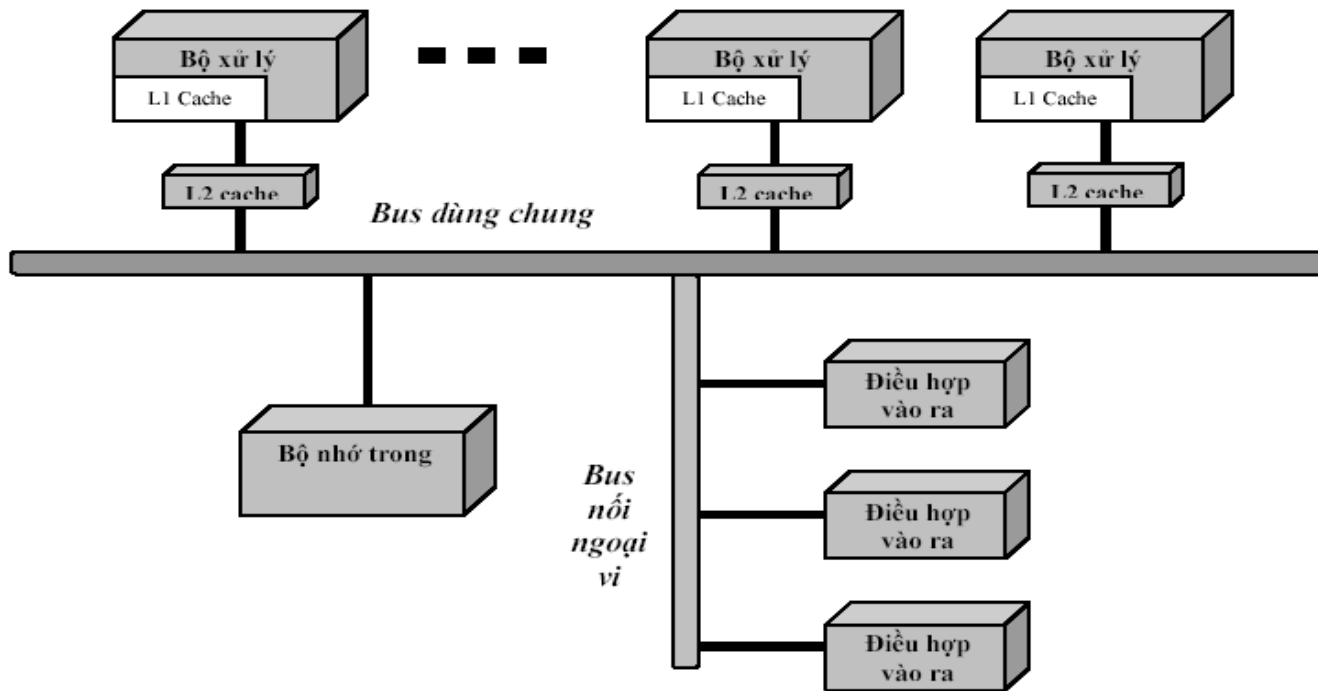
❖ SMP (Symmetric Multiprocessors):

- Bộ nhớ chia sẻ, các trao đổi thông qua bộ nhớ chia sẻ.
- Dùng chung hệ thống bus bộ nhớ.
- Thời gian truy cập bộ nhớ giống nhau cho các bộ xử lý



5. Máy tính song song (6)

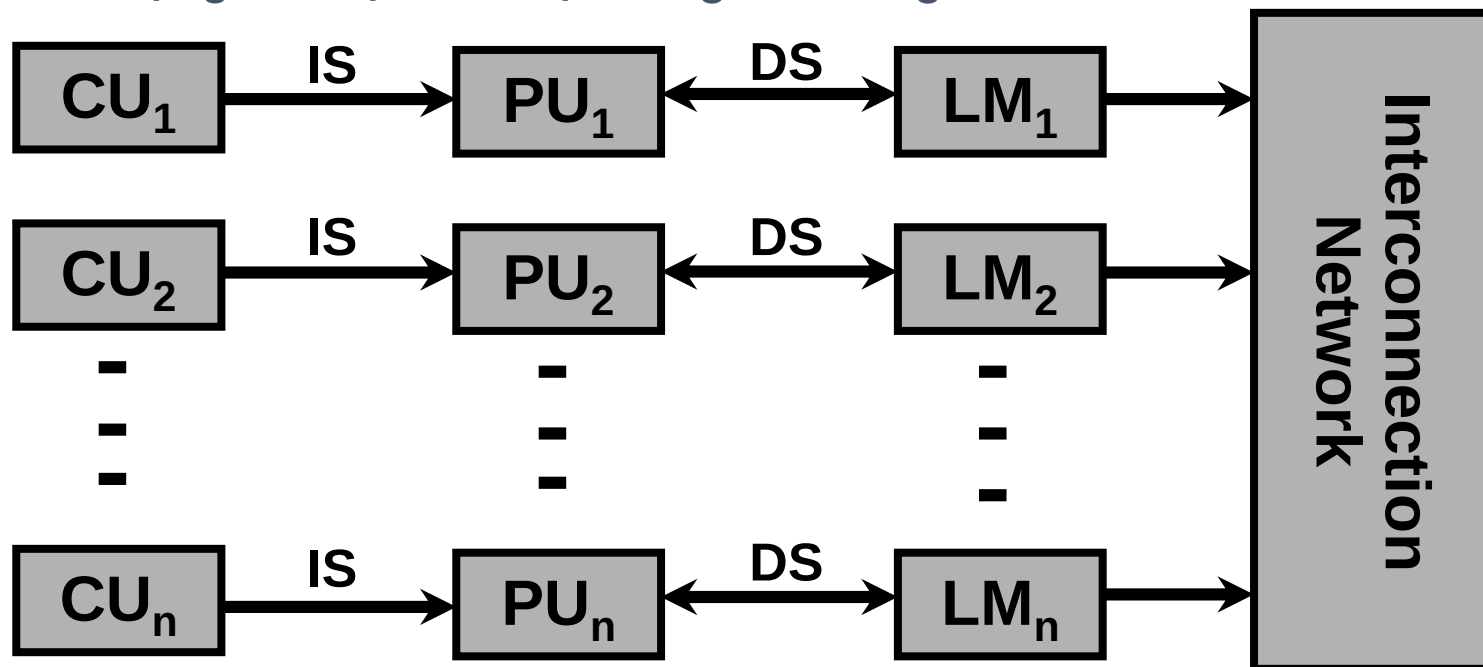
- NUMA (NonUniForm Memory Access)
- CC-NUMA (Cache-Coherent NUMA):



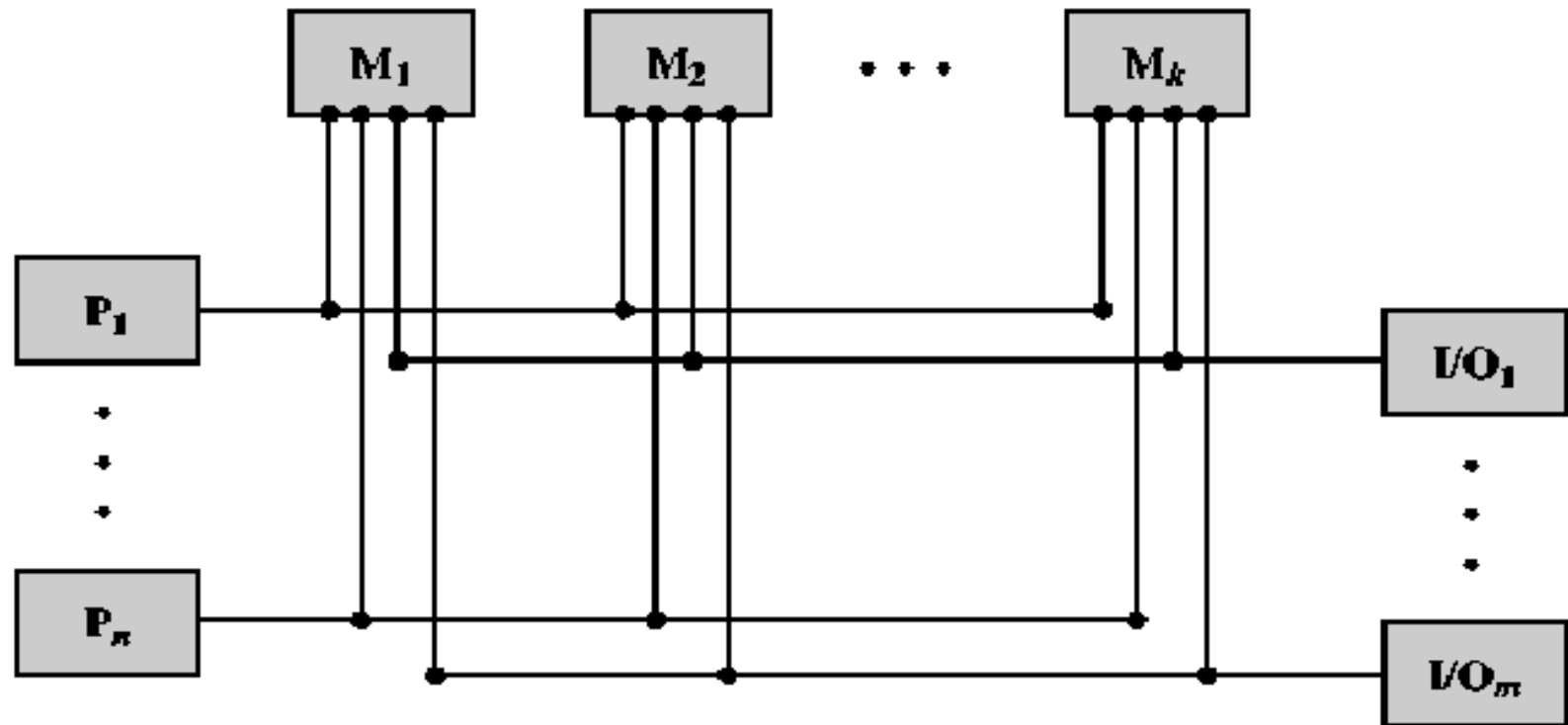
Hình III.9: Máy tính song song với bộ nhớ dùng chung, hệ thống bus dùng chung

5. Máy tính song song (7)

- Máy tính MIMD với bộ nhớ phân tán
 - Mỗi máy gồm có các nút,
 - Mỗi nút chứa một bộ xử lý, bộ nhớ, một vài ngã vào ra và một giao diện với hệ thống kết nối giữa các nút.

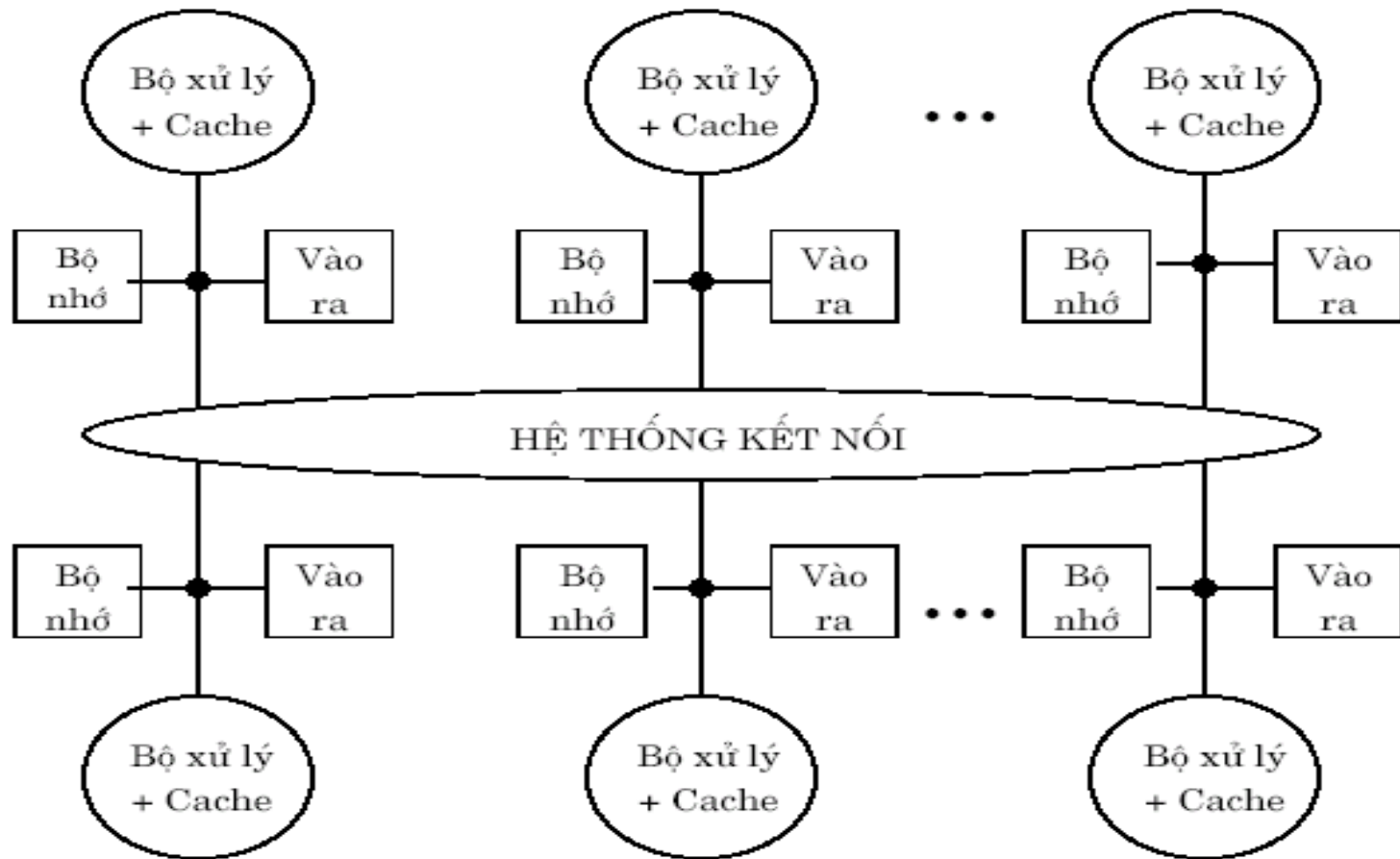


5. Máy tính song song (8)



Hình III.11: Tổ chức kết nối của máy tính song song có bộ nhớ phân tán

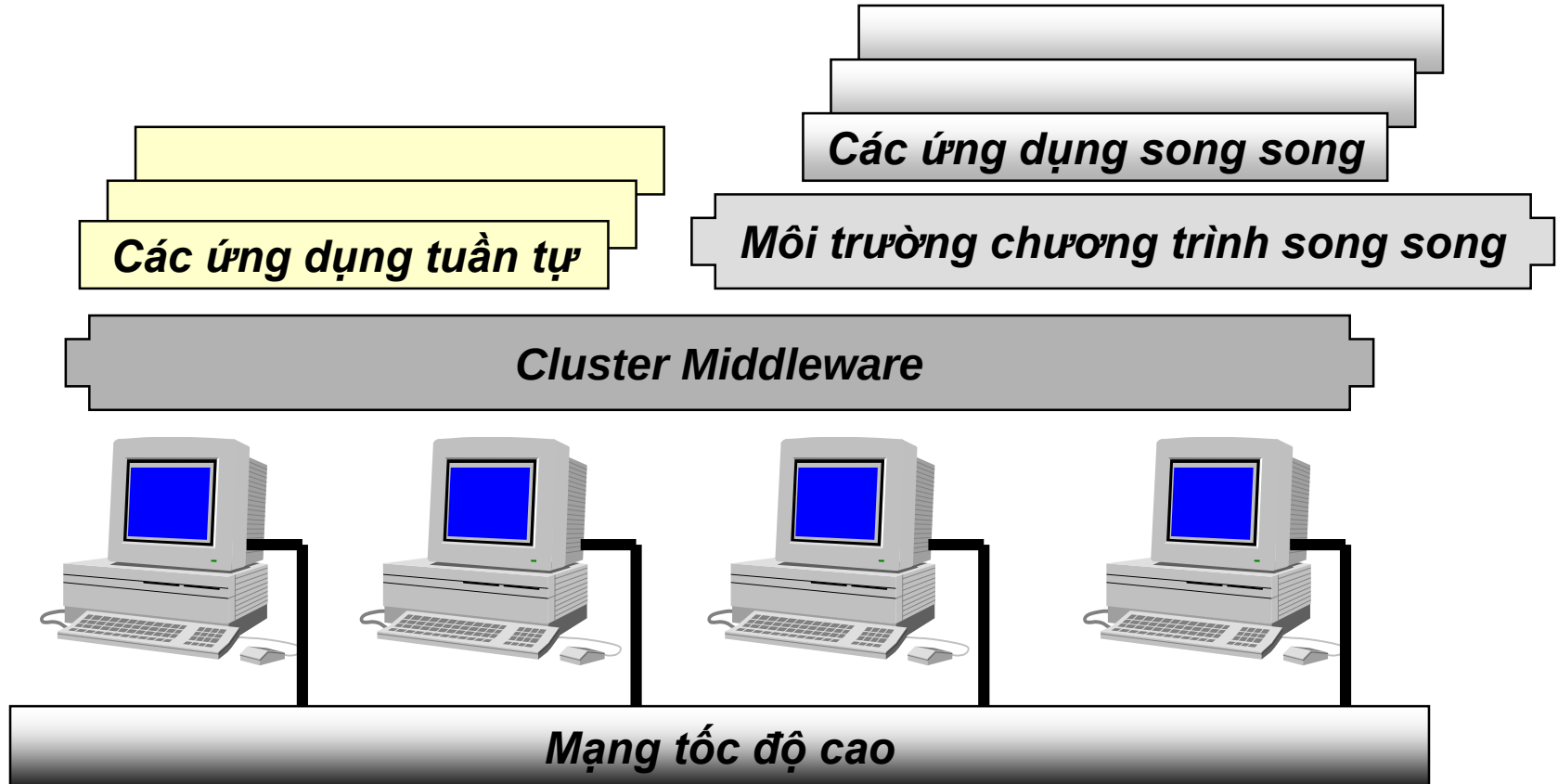
5. Máy tính song song (9)



Cấu trúc nền của một máy đa xử lý với bộ nhớ phân tán

4.4. Kiến trúc của các bộ xử lý tiên tiến (13)

5. Máy tính song song (10)



Kiến trúc máy tính Cluster

5. Máy tính song song (11)

- **Nhận xét:** Kiến trúc song song đang tiến triển chậm:
 - Do vấn đề phần mềm gặp nhiều khó khăn
 - Do các máy tính đa xử lý còn tiến chậm trong việc cải tiến giao diện với người sử dụng và tăng cường hiệu quả của chúng.
- **Hướng phát triển:** Có nhiều lý do để ta lạc quan vào tương lai của việc xử lý song song và của các máy đa xử lý:
 - Việc dùng xử lý song song bắt đầu được hiểu biết trong vài lĩnh vực, đặc biệt trong lĩnh vực tính toán khoa học và công nghệ. Trong các lĩnh vực này người ta luôn cần đến máy tính có tính năng cao hơn.
 - Hiện tại, người ta đã chấp nhận rằng một trong những cách hiệu quả nhất để chế tạo máy tính có tính năng cao hơn các máy đơn xử lý là chế tạo các máy tính đa xử lý.
 - Máy tính đa xử lý rất hiệu quả khi dùng cho đa chương trình. Đa chương trình được dùng chủ yếu cho các máy tính lớn và cho các máy phục vụ lớn.

4.5. Kiến trúc IA64 (1)

- Kiến trúc IA-64 là một kiến trúc mới được giới thiệu trong những năm gần đây. Kiến trúc này là sản phẩm của sự kết hợp nghiên cứu giữa hai công ty máy tính hàng đầu thế giới là Intel, HP (Hewlett Packard) và một số trường đại học.
- Kiến trúc IA-64 giới thiệu một sự khởi đầu mới quan trọng của kỹ thuật siêu vô hướng - kỹ thuật xử lý lệnh song song (EPIC: Explicitly Parallel Instruction Computing).
- Sản phẩm đầu tiên thuộc kiến trúc này là bộ xử lý ***Itanium***.

4.5. Kiến trúc IA64 (2)

Đặc trưng của kiến trúc IA-64:

- Cơ chế xử lý song song các lệnh mã máy (EPIC) thay vì các bộ xử lý song song như hệ thống đa bộ xử lý.
- Các lệnh dài hay rất dài (LIW hay VLIW).
- Các lệnh rẽ nhánh xác định (thay vì đoán các lệnh rẽ nhánh như các kiến trúc trước).
- Nạp trước các lệnh (theo sự suy đoán).

Số thanh ghi các bộ xử lý kiến trúc IA-64 là 256 thanh ghi.

Trong đó:

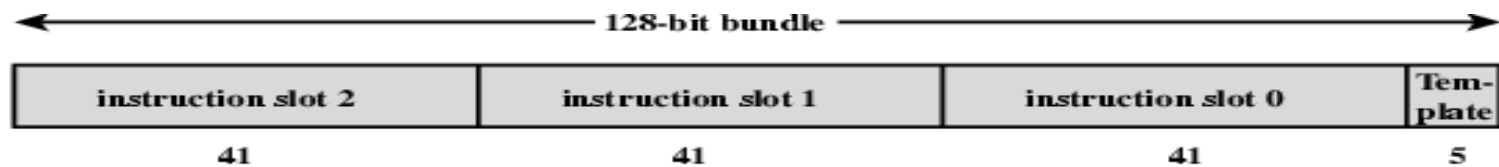
- 128 thanh ghi tổng quát 64 bit cho các tính toán số nguyên, luận lý
- 128 thanh ghi 82 bit cho các phép tính dấu chấm động và dữ liệu đồ họa
- 64 thanh ghi 1 bit thuộc tính để chỉ ra các thuộc tính lệnh đang thi hành.

4.5. Kiến trúc IA64 (3)

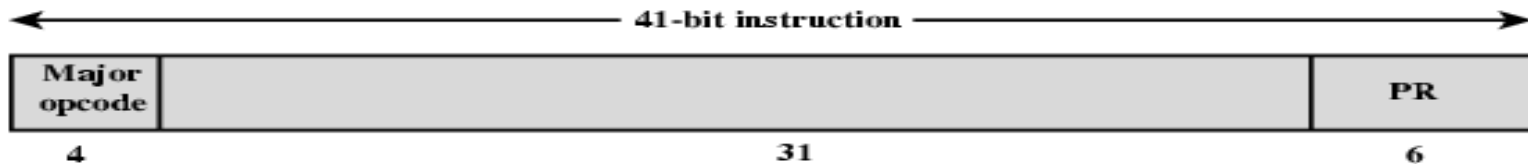
- Nhiều bộ thi hành lệnh: một máy tính có thể có tám hay nhiều hơn các bộ thi hành lệnh song song. Các bộ thi hành lệnh này được chia thành bốn kiểu:
 - Kiểu I (I-Unit): dùng xử lý các lệnh tính toán số nguyên, dịch, luận lý, so sánh, đa phương tiện.
 - Kiểu M (M-Unit): Nạp và lưu trữ giữa thanh ghi và bộ nhớ thêm vào một vài tác vụ ALU.
 - Kiểu B (B-Unit): Thực hiện các lệnh rẽ nhánh.
 - Kiểu F (F-Unit): Các lệnh tính toán số dấu chấm động

4.5. Kiến trúc IA64 (4)

Các định dạng lệnh:



(a) IA-64 bundle



(b) General IA-64 instruction format



(c) Typical IA-64 instruction format

PR = Predicate register

GR = General or floating-point register

Định dạng lệnh trong kiến trúc IA-64

